LibSort — Build Plan for Codex

Stack

- Backend: Django + Django REST Framework
- DB: SQLite (dev), Postgres-ready
- Frontend: React (Vite)
- QR: `qrcode` (Python, generation) + `html5-qrcode` (JS, scanning)
- Auth: Django's built-in auth, staff vs student roles via `is_staff` / a Profile model

Phase 1 — Backend foundation

1. `django-admin startproject libsort_backend`, create app `core`.
2. Models:
   - `Floor` (name)
   - `Section` (floor FK, name)
   - `Shelf` (section FK, code e.g. GF-SS-01, capacity, current_occupancy)
   - `Book` (title, author, isbn/book_id, category, shelf FK [assigned/home shelf], qr_code image field or generated on demand)
   - `Student` (name, student_id, contact — for demo/borrow records)
   - `BorrowRecord` (book FK, student FK, borrowed_date, due_date, returned_date nullable, status: borrowed/returned/overdue)
   - `ShelvingLog` (book FK, staff user FK, scanned_shelf FK, expected_shelf FK, result: correct/misplaced, timestamp)
3. Register all in Django admin — this alone satisfies "admin can add/edit shelves without rebuilding."
4. Migrate, create superuser.

Phase 2 — Seed data

5. Management command `seed_demo_data`:
   - Create 3 floors, 3 sections, 36 shelves per the LibSort doc structure (GF-SS-01..09, FF-ART-01..10, TF-ST-01..17).
   - Populate ~100 books: ~10-12 from the nursing journal PDFs (title/author/category="Nursing"), rest from a public book list (Open Library sample or a hardcoded seed list) spread across categories/shelves.
   - Create a handful of sample students and a few borrow records, including 1-2 deliberately overdue.

Phase 3 — Core API endpoints (DRF)

6. `GET /api/books/search?q=` — search by title/author/isbn/category. Returns title, availability, shelf_code, floor, section, exact location.
7. `GET /api/books/<id>/location` — staff-facing: shows registered shelf for a returned book.
8. `POST /api/scan-verify` — body: {book_id, scanned_shelf_code}. Compares to book's assigned shelf. Returns correct/misplaced + expected shelf if misplaced. Creates a ShelvingLog entry.
9. `GET /api/overdue` — list of overdue books + borrowers, staff-only.
10. `POST /api/overdue/<id>/remind` — stub notification (log it / print statement is fine for defense; mention "email/SMS integration point" in docs).
11. Shelf/floor/section CRUD — can just use Django admin, no need for custom API unless time allows.

Phase 4 — QR codes

12. On book creation, generate a QR encoding the book's shelf_code (or book_id) using `qrcode`, store as an ImageField or serve on demand via an endpoint.
13. Frontend scan screen uses `html5-qrcode` to read the camera feed, decodes the QR, sends the decoded value to `/api/scan-verify`.

Phase 5 — Frontend (React)

14. Pages:
    - **Student search** — search bar, results list with shelf code/floor/section/exact location.
    - **Staff scan-and-verify** — camera scan UI, shows result (green "correct" / red "MISPLACED BOOK — belongs on Shelf X").
    - **Staff overdue dashboard** — table of overdue books/borrowers, reminder button.
    - **Login** — basic staff login (students don't need accounts for search).
15. Keep styling minimal/clean — function over polish, this is a defense demo not a product launch.

Phase 6 — Polish for defense

16. Write a short README covering: architecture, how scan-input is abstracted (future hardware swap-in), how to run locally, how seed data was sourced.
17. Test the 6 test cases from the LibSort doc (T-01 to T-06) manually before defense day.

Notes for Codex

- Prioritize Phases 1-4 first — that's the functional core matching the 3 confirmed objectives (shelf map, scan-verify, evaluation-ready).
- Phase 5 can be minimal at first pass (plain forms), polish only if time remains.
- Do not add ISBN→Dewey external API lookups or RFID/Koha integration — explicitly out of scope per the LibSort requirements doc.